TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 2: XỬ LÝ DỮ LIỆU VỚI NUMPY & PANDAS

Nền tảng thao tác dữ liệu cho Trí tuệ nhân tạo

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày: 22/9/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả và dán vào khung viền xám đứt nét (🖼️)
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Giải thích được vì sao AI cần dữ liệu dạng mảng số (NumPy array) thay vì list Python thuần
- Thực hiện được reshape, broadcasting, nhân ma trận cơ bản với NumPy
- Tạo và xem nhanh dữ liệu bảng (DataFrame) bằng Pandas: head, describe, info
- Lọc, nhóm (groupby) và xử lý giá trị thiếu trong dữ liệu bằng Pandas
- Tự đặt và trả lời câu hỏi thống kê trên một bộ dữ liệu thật

TIẾT 1 — TRẢI NGHIỆM: KHÁM PHÁ DỮ LIỆU VỚI PANDAS & NUMPY

Mở notebook "Demo_Buoi2.ipynb" (đã có sẵn toàn bộ code bên dưới). Với mỗi demo, bạn chỉ cần: copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo 1: Tạo và khám phá bảng điểm lớp bằng Pandas

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import pandas as pd

diem_lop = {
    "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc"],
    "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0],
    "van":  [7.0, 8.0, 8.5, 6.5, 7.5, 9.0],
    "anh":  [9.0, 7.0, 8.0, 6.0, 6.5, 8.5],
}

df = pd.DataFrame(diem_lop)
print(df.head())
print(df.describe())
```

💭 **Suy nghĩ: describe() vừa in ra những con số gì? Chúng nói lên điều gì về lớp học này?**

df.describe() chỉ thống kê cột số (toan, van, anh), không thống kê cột ten.
Mặt bằng khá (trung bình 7.4–7.8), không ai quá kém, không ai 10. Văn ổn định nhất; Toán phân hóa rõ hơn.

df.head() chỉ in 5 dòng đầu nên **Phúc** không hiện, nhưng describe() vẫn tính đủ **6** bạn (count = 6).

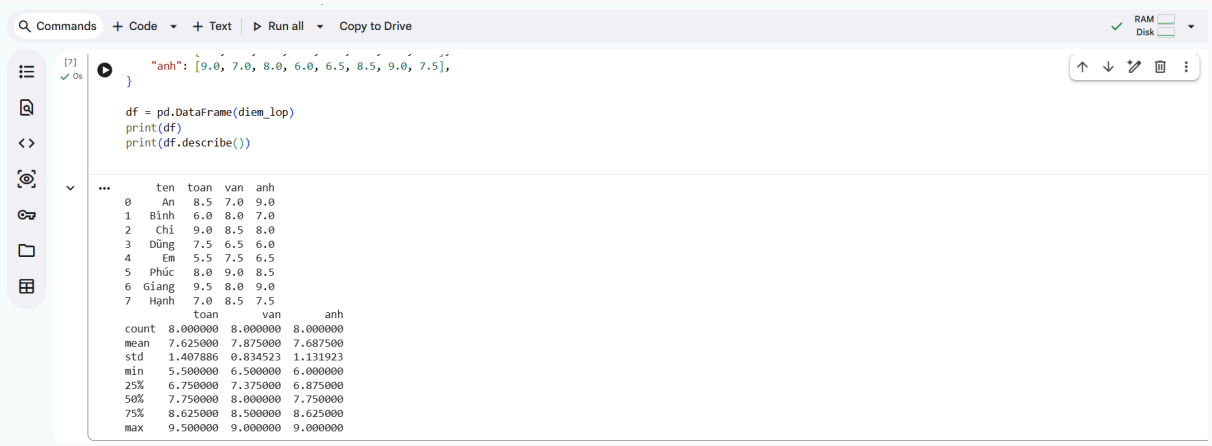
🔧 Cải tiến: Hãy thêm ít nhất 2 sinh viên mới vào dictionary diem_lop (thêm tên và điểm 3 môn), chạy lại và xem describe() thay đổi ra sao:

```
import pandas as pd

diem_lop = {
    "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc", "Giang", "Hạnh"],
    "toan": [8.5, 6.0, 9.0, 7.5, 5.5, 8.0, 9.5, 7.0],
    "van": [7.0, 8.0, 8.5, 6.5, 7.5, 9.0, 8.0, 8.5],
    "anh": [9.0, 7.0, 8.0, 6.0, 6.5, 8.5, 9.0, 7.5],
}

df = pd.DataFrame(diem_lop)
print(df)
print(df.describe())
```

🌾 Thu hoạch: chụp màn hình kết quả head()/describe() sau khi thêm sinh viên mới:



Demo 2: Mảng NumPy cơ bản

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import numpy as np

diem_toan = np.array([8.5, 6.0, 9.0, 7.5, 5.5, 8.0])
print("Trung bình:", diem_toan.mean())
print("Độ lệch chuẩn:", diem_toan.std())
print("Điểm cao nhất:", diem_toan.max())
```

💭 **Suy nghĩ: mean() và std() cho bạn biết điều gì khác nhau về dữ liệu?**

mean() = mặt bằng điểm. std() = điểm lệch nhau nhiều hay ít. Mean cao chưa chắc lớp đều; std mới nói độ phân hóa.

🔧 Cải tiến: Hãy tự tạo một mảng NumPy mới từ điểm môn văn hoặc anh (xem lại Demo 1), rồi tính thêm điểm thấp nhất (min()) và trung vị (median()):

```
import numpy as np

diem_van = np.array([7.0, 8.0, 8.5, 6.5, 7.5, 9.0])

print("Trung bình:", diem_van.mean())
print("Độ lệch chuẩn:", diem_van.std())
print("Điểm cao nhất:", diem_van.max())
print("Điểm thấp nhất:", diem_van.min())
print("Trung vị:", np.median(diem_van))
```

🌾 Thu hoạch: chụp màn hình kết quả các phép tính trên mảng mới:



Demo 3: Vì sao cần NumPy? So sánh tốc độ

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import numpy as np
import time

n = 1_000_000
danh_sach = list(range(n))
mang = np.arange(n)

start = time.time()
tong_binh_phuong_loop = sum(x**2 for x in danh_sach)
print("Thời gian vòng lặp Python:", time.time() - start, "giây")

start = time.time()
tong_binh_phuong_np = np.sum(mang**2)
print("Thời gian NumPy:", time.time() - start, "giây")
```

💭 **Suy nghĩ: Cách nào nhanh hơn? Nhanh hơn khoảng bao nhiêu lần?**

NumPy nhanh hơn vòng lặp Python. Với n = 1_000_000 thường nhanh khoảng vài chục lần (máy này ~20 lần). Lý do: Python cộng từng phần tử; NumPy tính cả khối bằng C.

🔧 Cải tiến: Hãy đổi n thành 10.000.000 rồi chạy lại — khoảng cách tốc độ có thay đổi không?

```
import numpy as np
import time

n = 10_000_000
```

```
danh_sach = list(range(n))
mang = np.arange(n)

start = time.time()
tong_binh_phuong_loop = sum(x**2 for x in danh_sach)
print("Thời gian vòng lặp Python:", time.time() - start, "giây")

start = time.time()
tong_binh_phuong_np = np.sum(mang**2)
print("Thời gian NumPy:", time.time() - start, "giây")
```

🌾 Thu hoạch: chụp màn hình so sánh thời gian chạy với giá trị n mới:



```
import numpy as np
import time

n = 10_000_000
danh_sach = list(range(n))
mang = np.arange(n)

start = time.time()
tong_binh_phuong_loop = sum(x**2 for x in danh_sach)
print("Thời gian vòng lặp Python:", time.time() - start, "giây")

start = time.time()
tong_binh_phuong_np = np.sum(mang**2)
print("Thời gian NumPy:", time.time() - start, "giây")
```

```
Thời gian vòng lặp Python: 1.1327123641967773 giây
Thời gian NumPy: 0.0368196964263916 giây
```

Câu hỏi thảo luận: Vì sao AI/Machine Learning xử lý dữ liệu cực lớn lại cần NumPy/Pandas thay vì list/dict thuần Python?

AI/ML làm việc với **hàng triệu–tỷ số** (ảnh, token, bảng đặc trưng). list/dict của Python giữ từng phần tử như một object riêng, vòng for chạy từng bước trên interpreter → chậm và tốn RAM.

NumPy/Pandas thì ngược lại:

Tính theo khối (vectorization): một lệnh X @ W hay df["gia"].mean() chạy bằng C/Fortran, không lặp Python từng dòng. Demo 3 của bạn đã thấy: 10 triệu phần tử, vòng lặp ~1.13 s, NumPy ~0.037 s (~30 lần).

Bộ nhớ gọn: mảng kiểu đồng nhất (float32/float64), không phải box từng số thành object. Cùng một tensor, list Python dễ nặng gấp nhiều lần.

Đúng "hình dạng" dữ liệu ML: ma trận, tensor, bảng có cột. NumPy có shape/broadcast; Pandas có lọc, groupby, join — đúng việc chuẩn bị dữ liệu trước khi đưa vào model.

Cầu nối hệ sinh thái: scikit-learn, PyTorch, TensorFlow, matplotlib đều nhận/trả ndarray hoặc DataFrame. Viết list thuần là tự cắt mình khỏi stack đó.

list/dict hợp dữ liệu nhỏ, linh hoạt; AI cần nhanh + ít RAM + tính toán mảng. Đó là chỗ NumPy/Pandas được thiết kế cho.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

Mảng NumPy khác list Python như thế nào?

List Python chứa lẫn kiểu, mỗi phần tử là một object, cộng/nhân phải for từng cái → linh hoạt nhưng chậm, tốn RAM.

Mảng NumPy chứa **cùng một kiểu** (toàn float chẳng hạn), nằm liền nhau trong bộ nhớ, có shape. Cộng, nhân, mean(), std() chạy cả khối. Đổi lại: ít "tự do" hơn list (thêm phần tử giữa mảng không tiện bằng append).

Vector hóa (vectorization) là gì? Broadcasting là gì?

Vector hóa: viết phép toán trên **cả mảng**, không viết vòng lặp.

mang ** 2 thay vì for x in danh_sach: x**2. Máy tính làm phía dưới bằng C nên nhanh (đúng như Demo 3).

Broadcasting: NumPy **tự kéo** hai mảng khác kích thước cho khớp để tính được. Ví dụ cộng vector [1, 2, 3] vào từng hàng của ma trận 3 cột mà không cần nhân bản tay. Sai shape thì báo lỗi — đó là lúc broadcasting không áp dụng được.

DataFrame và Series trong Pandas khác nhau như thế nào?

Series = **một cột** (hoặc một hàng): có index + giá trị. Giống một danh sách được gắn nhãn.

DataFrame = **bảng**: nhiều Series đặt cạnh nhau, chung index. Cột ten, toan, van, anh trong Demo 1 chính là DataFrame; riêng cột toan lấy ra là Series.

Làm việc hàng ngày: xem cả lớp → DataFrame; tính trung bình một môn → Series.

Vì sao dữ liệu cần được 'làm sạch' (xử lý giá trị thiếu…) trước khi đưa vào mô hình AI?

Mô hình chỉ hiểu số, và giả định ô nào cũng có giá trị hợp lệ.

Thiếu điểm, ghi "N/A", nhập nhầm 85 thành 8..5, trùng tên học sinh… sẽ làm mean() sai, hoặc code sập, hoặc model học nhầm.

Làm sạch = quyết định thiếu thì điền/bỏ, sửa kiểu dữ liệu, loại nhiễu. Bỏ bước này thì model giỏi đến mấy cũng học từ dữ liệu hỏng.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: NUMPY & PANDAS

📋 Nội dung có sẵn — NumPy: reshape, broadcasting, nhân ma trận. Copy và chạy:

```
import numpy as np

diem_3_mon = np.array([8.5, 7.0, 9.0, 6.0, 7.5, 8.0, 6.5, 8.5, 9.0, 7.0, 8.0,
6.5])
bang_diem = diem_3_mon.reshape(4, 3)   # 4 sinh viên, 3 môn
```

```
print(bang_diem)

# Cộng thêm 0.5 điểm khuyến khích cho tất cả (broadcasting)
bang_diem_moi = bang_diem + 0.5
print(bang_diem_moi)

# Điểm trung bình mỗi sinh viên (theo hàng)
print("Điểm trung bình mỗi sinh viên:", bang_diem.mean(axis=1))
```

```
diem = np.array([[8, 7, 9], [6, 8, 7]])    # 2 sinh viên, 3 môn
trong_so = np.array([0.3, 0.3, 0.4])        # trọng số môn học

diem_tong_ket = diem.dot(trong_so)
print("Điểm tổng kết:", diem_tong_ket)
```

💭 **Suy nghĩ: axis=1 trong mean(axis=1) có ý nghĩa gì? Nếu đổi thành axis=0 kết quả sẽ khác ra sao?**

axis=1 = tính theo hàng → trung bình từng sinh viên (3 môn).

Đổi axis=0 = tính theo cột → trung bình từng môn (cả lớp).

🔧 Cải tiến: (1) đổi số điểm khuyến khích trong broadcasting (VD: +1.0 thay vì +0.5); (2) đổi trọng số môn học trong phép nhân ma trận sao cho tổng vẫn bằng 1 (VD: 0.2/0.3/0.5):

```
import numpy as np

diem_3_mon = np.array([8.5, 7.0, 9.0, 6.0, 7.5, 8.0, 6.5, 8.5, 9.0, 7.0, 8.0, 6.5])
bang_diem = diem_3_mon.reshape(4, 3)
print(bang_diem)

# (1) khuyến khích +1.0 thay vì +0.5
bang_diem_moi = bang_diem + 1.0
print(bang_diem_moi)

print("Điểm trung bình mỗi sinh viên:", bang_diem.mean(axis=1))

# (2) trọng số mới, tổng vẫn = 1
diem = np.array([[8, 7, 9], [6, 8, 7]])
trong_so = np.array([0.2, 0.3, 0.5])
diem_tong_ket = diem.dot(trong_so)
print("Điểm tổng kết:", diem_tong_ket)
```

🌾 Thu hoạch: chụp màn hình kết quả sau khi cải tiến:



📋 Nội dung có sẵn — Pandas: lọc dữ liệu, groupby, xử lý giá trị thiếu. Copy và chạy:

```
import pandas as pd
```

```
import numpy as np

diem_lop2 = {
    "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc"],
    "lop": ["A1", "A1", "A2", "A2", "A1", "A2"],
    "toan": [8.5, 6.0, 9.0, np.nan, 5.5, 8.0],
    "van":  [7.0, 8.0, 8.5, 6.5, np.nan, 9.0],
}
df2 = pd.DataFrame(diem_lop2)

# Lọc sinh viên có điểm toán >= 7
gioi_toan = df2[df2["toan"] >= 7]
print(gioi_toan)

# Nhóm theo lớp, tính điểm trung bình môn toán
print(df2.groupby("lop")["toan"].mean())

# Kiểm tra và xử lý giá trị thiếu
print(df2.isnull().sum())
df2_filled = df2.fillna(df2.mean(numeric_only=True))
print(df2_filled)
```

💭 **Suy nghĩ: fillna đã điền giá trị gì vào các ô bị thiếu? Cách điền này có hợp lý không, vì sao?**

fillna(df2.mean(...)) điền **trung bình cột**: Toán thiếu (Dũng) ← **7.4**, Văn thiếu (Em) ← **7.8**.

Hợp lý **hơn điền 0** vì 0 coi như điểm liệt, kéo mean/groupby lệch nặng. Điền trung bình giữ mặt bằng lớp, nhưng cũng "bịa" điểm — không phải điểm thật.

🔧 Cải tiến: (1) đổi ngưỡng lọc từ >= 7 sang >= 8; (2) nhóm theo lớp và tính điểm trung bình môn văn thay vì toán; (3) thử fillna bằng giá trị 0 thay vì trung bình và so sánh:

```
import pandas as pd
import numpy as np

diem_lop2 = {
   "ten": ["An", "Bình", "Chi", "Dũng", "Em", "Phúc"],
   "lop": ["A1", "A1", "A2", "A2", "A1", "A2"],
   "toan": [8.5, 6.0, 9.0, np.nan, 5.5, 8.0],
   "van": [7.0, 8.0, 8.5, 6.5, np.nan, 9.0],
}
df2 = pd.DataFrame(diem_lop2)

# (1) lọc toán >= 8
print(df2[df2["toan"] >= 8])

# (2) groupby lớp — trung bình Văn
print(df2.groupby("lop")["van"].mean())

print(df2.isnull().sum())

# (3) so sánh hai cách điền
print(df2.fillna(df2.mean(numeric_only=True)))
```

```
print(df2.fillna(0))
```

🌾 Thu hoạch: chụp màn hình kết quả sau khi cải tiến:



```
    ten lop  toan  van
0    An  A1   8.5  7.0
2   Chi  A2   9.0  8.5
5  Phúc  A2   8.0  9.0
lop
A1    7.5
A2    8.0
Name: van, dtype: float64
ten     0
lop     0
toan    1
van     1
dtype: int64
    ten lop  toan  van
0    An  A1   8.5  7.0
1  Bình  A1   6.0  8.0
2   Chi  A2   9.0  8.5
3  Dũng  A2   7.4  6.5
4    Em  A1   5.5  7.8
5  Phúc  A2   8.0  9.0
    ten lop  toan  van
0    An  A1   8.5  7.0
1  Bình  A1   6.0  8.0
2   Chi  A2   9.0  8.5
3  Dũng  A2   0.0  6.5
4    Em  A1   5.5  0.0
5  Phúc  A2   8.0  9.0
```

TIẾT 4 — ÁP DỤNG MỞ RỘNG: PHÂN TÍCH DỮ LIỆU BÁN HÀNG

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy đoạn code phân tích dữ liệu bán hàng sau (đã có sẵn 1 câu hỏi mẫu đã trả lời):

```
import pandas as pd

ban_hang = {
     "san_pham": ["Gạo", "Đường", "Sữa", "Nước mắm", "Mì gói", "Trứng", "Dầu ăn", "Rau"],
      "danh_muc": ["Lương thực", "Thực phẩm", "Thực phẩm", "Gia vị", "Lương thực", "Thực phẩm", "Gia vị", "Rau củ"],
    "gia_ban": [18000, 22000, 30000, 35000, 4000, 3000, 45000, 12000],
    "so_luong_ban": [120, 80, 60, 90, 200, 300, 40, 150],
}
df3 = pd.DataFrame(ban_hang)
df3["doanh_thu"] = df3["gia_ban"] * df3["so_luong_ban"]
print(df3)

# Câu hỏi mẫu đã có sẵn: Danh mục nào có tổng doanh thu cao nhất?
print(df3.groupby("danh_muc")["doanh_thu"].sum().sort_values(ascending=False)
)
```

✅ Chạy code trên và hiểu rõ câu hỏi mẫu đã được trả lời như thế nào

🔧 Cải tiến: Cả nhóm cùng tự đặt và trả lời thêm ít nhất 5 câu hỏi thống kê khác trên df3 (gợi ý: sản phẩm bán chạy nhất, giá trung bình theo danh mục, sản phẩm có giá trên mức trung bình…), dựa theo mẫu groupby/lọc/sort_values ở trên:

```
import pandas as pd

ban_hang = {
   "san_pham": ["Gạo", "Đường", "Sữa", "Nước mắm", "Mì gói", "Trứng", "Dầu ăn", "Rau"],
    "danh_muc": ["Lương thực", "Thực phẩm", "Thực phẩm", "Gia vị", "Lương thực", "Thực phẩm", "Gia vị", "Rau củ"],
   "gia_ban": [18000, 22000, 30000, 35000, 4000, 3000, 45000, 12000],
```

```
    "so_luong_ban": [120, 80, 60, 90, 200, 300, 40, 150],
}
df3 = pd.DataFrame(ban_hang)
df3["doanh_thu"] = df3["gia_ban"] * df3["so_luong_ban"]
print(df3)

# Câu hỏi mẫu đã có sẵn: Danh mục nào có tổng doanh thu cao nhất?
print(df3.groupby("danh_muc")["doanh_thu"].sum().sort_values(ascending=False))

# 1. Sản phẩm bán chạy nhất (số lượng)?
print(df3.sort_values("so_luong_ban", ascending=False)[["san_pham", "so_luong_ban"]].head(1))

# 2. Sản phẩm doanh thu cao nhất / thấp nhất?
print(df3.sort_values("doanh_thu", ascending=False)[["san_pham", "doanh_thu"]])

# 3. Giá trung bình theo danh mục?
print(df3.groupby("danh_muc")["gia_ban"].mean())

# 4. Sản phẩm có giá trên mức trung bình cả bảng?
print(df3[df3["gia_ban"] > df3["gia_ban"].mean()][["san_pham", "gia_ban"]])

# 5. Danh mục nào bán được nhiều món nhất (tổng số lượng)?
print(df3.groupby("danh_muc")["so_luong_ban"].sum().sort_values(ascending=False))
```

🌾 Thu hoạch: chụp màn hình kết quả chạy 5 câu hỏi của nhóm:

```
  san_pham  so_luong_ban
5    Trứng          300
...
   san_pham  doanh_thu
3  Nước mắm    3150000
0       Gạo    2160000
2       Sữa    1800000
7       Rau    1800000
6    Dầu ăn    1800000
1     Đường    1760000
5     Trứng     900000
4    Mì gói     800000
danh_muc
Gia vị        40000.000000
Lương thực    11000.000000
Rau củ        12000.000000
Thực phẩm     18333.333333
Name: gia_ban, dtype: float64
   san_pham  gia_ban
1     Đường    22000
2       Sữa    30000
3  Nước mắm    35000
6    Dầu ăn    45000
danh_muc
Thực phẩm     440
Lương thực    320
Rau củ        150
Gia vị        130
Name: so_luong_ban, dtype: int64
```

Câu hỏi thú vị nhất mà nhóm bạn đặt ra và phát hiện được:

Câu hỏi: Sản phẩm bán chạy nhất có phải là sản phẩm đem lại doanh thu cao nhất không?

Phát hiện: Không. Trứng bán nhiều nhất (300) nhưng doanh thu chỉ 900.000 vì giá 3.000. Nước mắm bán có 90 chai mà thu 3.150.000 — cao nhất bảng. Cùng kiểu đó, danh mục Thực phẩm bán nhiều món nhất (440) nhưng tổng thu thua Gia vị (130 món, 4.950.000).

Kết luận: số lượng cao chưa chắc giàu; giá nhân số lượng mới quyết định. Nhìn mỗi "bán chạy" dễ hiểu sai mặt hàng nào đang nuôi cửa hàng.

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Tôi đã chạy bảng điểm lớp bằng Pandas (head, describe), mảng NumPy (mean, std, min, median), so sánh tốc độ list Python với NumPy (n = 1 triệu và 10 triệu), reshape / broadcasting / nhân trọng số, lọc–groupby–fillna trên điểm có ô trống, và bảng bán hàng (doanh thu, groupby danh mục). Quan sát: describe cho count/mean/std/min/max; Colab của tôi với n = 10 triệu mất khoảng 1.13 giây (Python) và 0.037 giây (NumPy); Gia vị có tổng doanh thu cao nhất (4.950.000); điền thiếu bằng trung bình ra Dũng Toán 7.4, Em Văn 7.8, còn điền 0 thì thành 0.

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Một phần đúng dự đoán: NumPy phải nhanh hơn, groupby cho biết danh mục nào thu nhiều. Bất ngờ: Trứng bán chạy nhất (300) nhưng doanh thu thấp; Gia vị ít món lại thu nhiều nhất. fillna trung bình và fillna 0 cho hai bảng trông rất khác nhau dù cùng chỗ trống. Chưa thật rõ lúc nào nên điền trung bình cả khối, lúc nào nên điền theo từng lớp. Std của Pandas và NumPy cũng lệch nhau một chút vì khác công thức.

3. KHÁI QUÁT HÓA — NumPy và Pandas giúp xử lý dữ liệu khác gì so với dùng list/dict thuần Python? (diễn đạt bằng lời của chính mình)

List/dict giữ từng phần tử như object riêng, tính bằng vòng for nên chậm và khó xem cả bảng. NumPy là mảng cùng kiểu, tính cả khối (cộng 0.5 cho mọi ô, nhân ma trận trọng số). Pandas là bảng có tên cột: lọc toán >= 8, groupby lớp, điền ô trống. Cùng một việc "trung bình theo nhóm", Pandas một dòng; list phải tự viết vòng lặp.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu gặp một bộ dữ liệu mới, tôi sẽ bắt đầu khám phá nó như thế nào? Tôi còn thắc mắc điều gì?

Gặp dữ liệu mới tôi sẽ: in head và shape, describe, đếm isnull, xem kiểu cột, rồi mới lọc / groupby / vẽ thêm cột (như doanh thu = giá * số lượng). Không điền 0 bừa vào chỗ thiếu. Không có thắc mắc gì thêm.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — 5

✅ Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ code cải tiến, ảnh và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.